

Recuperatorio Primer Parcial de Programación Imperativa

22/06/2026

❖ **Condición mínima de aprobación: Sumar 5 (cinco) puntos.**

Ejercicio 1 (3 puntos)

Escribir una función que reciba un string, que puede contener cualquier carácter, y elimine del mismo cualquier aparición repetida de cada letra del alfabeto inglés (sin distinguir entre minúsculas y mayúsculas), dejando sólo la primera aparición.

Ejemplos:

```
int main(void) {
    char s1[] = "Abracadabra!!! Es lo que yo siempre digo";
    eliminaConsecutivosAlfabeticos(s1);
    assert(!strcmp(s1, "Abrcd!!! Es lo qu y imp g"));

    puts("OK!");
    return 0;
}
```

Ejercicio 2 (3,5 puntos)

Implementar una función que reciba tres matrices de números enteros:

- A de dimensión $F1 \times C1$
- B de dimensión $C1 \times C2$
- C de dimensión $F1 \times C2$

La función debe retornar 1 si la matriz C es exactamente el producto matricial de A por B, o 0 (cero) en caso contrario.

La función debe verificar que cada elemento de C coincida con el valor obtenido al realizar la multiplicación de matrices.

La función recibe:

- f1: cantidad de filas de la matriz A.
- c1: cantidad de columnas de la matriz A y cantidad de filas de la matriz B.
- c2: cantidad de columnas de la matriz B.
- A: primera matriz del producto.
- B: segunda matriz del producto.
- C: matriz que se desea verificar como resultado del producto $A \times B$.

```
int main(void) {

    int A[][3] = { {1, 2, 3},
                   {4, 5, 6} };
    int B[][2] = { {7, 8},
                   {9, 10},
                   {11, 12} };

    int Ccorrecta[][2] = { {58, 64},
                           {139, 154} };
```

Nombre y Apellido: N° Legajo:

```
int Cincorrecta[][2] = { {58, 64},
                        {139, 155} };

assert(esProducto(2, 3, 2, A, B, Ccorrecta) == 1);
assert(esProducto(2, 3, 2, A, B, Cincorrecta) == 0);

puts("OK!");
return 0;
}
```

Ejercicio 3 (3,5 puntos)

Escribir una función que reciba un vector de enteros y una matriz de unsigned char, donde se asegura que cada número está entre 0 y 9 (no se debe validar).

La función debe retornar cuántos números del vector se pueden formar con cada "fila" de la matriz (ver programa de ejemplo)

Se espera que el vector tenga 10 veces más elementos que la matriz. Por ejemplo si la matriz tiene 10 filas, el vector tendrá aproximadamente 100 números

Ejemplo (de funcionamiento, no de la relación de elementos entre matriz y vector)

```
int countNumbers(unsigned int cols, unsigned int rows,
unsigned char matrix[][cols],
unsigned int v[], unsigned int dim);

int main(void) {
    unsigned char m1[][5] = {
        {1, 2, 3, 4, 5},    // representa el 12345
        {6, 7, 8, 9, 0},    // representa el 67890
        {0, 0, 1, 2, 4},    // 124
        {0, 9, 1, 3, 2},    // 9132
    };

    unsigned int v[] = { 124, 124, 4, 33, 124 };
    assert(countNumbers(5, 4, m1, v, 5) == 3); // El 124 aparece 3 veces
    assert(countNumbers(5, 0, m1, v, 5) == 0); // 0 filas en la matriz
    assert(countNumbers(5, 4, m1, v, 0) == 0); // 0 elementos en v

    unsigned int v2[] = { 9132, 100, 67890, 100, 4, 3, 1, 1, 2, 3, 4, 5, 6, 7, 101 };
    assert(countNumbers(5, 4, m1, v2, 15) == 2); // El 9132 y el 67890

    unsigned char m2[][7] = {
        {0, 0, 0, 0, 1, 2, 4},    // representa el 124
        {1, 1, 6, 7, 8, 9, 0},    // representa el 1167890
        {0, 0, 0, 0, 1, 2, 4},    // 124
        {1, 0, 0, 9, 1, 3, 2},    // 1009132
    };

    assert(countNumbers(7, 4, m2, v, 5) == 3); // El 124 aparece 3 veces en v

    puts("OK");
    return 0;
}
```